

Ans1. Operating System: Definition, Goals, and Functions

An Operating System (OS) is fundamental system software that acts as an intermediary between user applications and computer hardware. Without an OS, each application would need to directly control hardware components such as the CPU, memory, disk, and I/O devices, which is complex and error-prone. The OS provides abstractions such as files, processes, virtual memory, and device interfaces to simplify application development and execution.

The two broad goals of an OS are:

1. **Convenience:** Make the system easier to use for users and programmers by providing a stable and friendly execution environment.
2. **Efficiency:** Manage hardware resources optimally so that CPU, memory, and I/O devices are utilized effectively with minimal waste.

Core functions of an OS include:

- **Process management:** creating, scheduling, synchronizing, and terminating processes.
- **Memory management:** allocation/deallocation, protection, paging/segmentation, virtual memory.
- **File system management:** organize data in files/directories with access permissions.
- **Device and I/O management:** coordinate communication with hardware via drivers and buffering.
- **Security and protection:** authentication, authorization, process isolation.
- **User interface:** command-line or graphical environment. Thus, the OS is both a resource manager and a control program.
-

Ans2. Process vs Thread

A **process** is an independent program in execution. It has its own address space, code, data, stack, and OS-managed resources. Process isolation improves reliability because one process crash generally does not directly corrupt another process.

A **thread** is the smallest unit of execution within a process. Multiple threads in the same process share code segment, data segment, and open resources, but each thread has its own stack and CPU register state. This sharing allows faster communication and reduced memory usage.

Why threads are called lightweight:

- Thread creation is cheaper than process creation.
- Context switching between threads is faster than between processes.
- Inter-thread communication is simpler because memory is shared.

Example: A web browser can be one process with multiple threads:

- UI thread,
- network thread,

- rendering thread. If implemented as separate processes, communication overhead would be higher.

In summary, processes provide stronger isolation, while threads provide better concurrency and efficiency.

Ans3. CPU Scheduling: FCFS vs Round Robin

CPU scheduling is the OS mechanism that selects which ready process should execute on the CPU next. Since many processes compete for limited CPU time, scheduling directly affects system performance and user experience.

FCFS (First-Come, First-Serve):

- Non-preemptive scheduling.
- Processes execute in order of arrival.
- Easy to implement and predictable.
- Major drawback: **convoy effect**, where short jobs wait behind a long job, increasing average waiting and turnaround time.

Round Robin (RR):

- Preemptive scheduling.
- Each process receives CPU for a fixed **time quantum**.
- If not finished, process is moved to end of ready queue.
- Good for interactive/time-sharing systems.
- Improves response time and fairness.
- Drawback: too small quantum increases context-switch overhead; too large quantum behaves like FCFS.

Comparison:

- **Fairness:** $RR > FCFS$
- **Response time:** RR generally better
- **Overhead:** FCFS lower, RR higher due to frequent context switches
- **Suitability:** FCFS for simple batch workloads; RR for multi-user interactive systems.

Ans4. Deadlock and Coffman Conditions

A deadlock is a state in which a group of processes are permanently blocked because each process is waiting for a resource held by another process in the group. As a result, no process can proceed.

Coffman's four necessary conditions:

1. **Mutual Exclusion:** At least one resource is non-shareable (e.g., printer).
2. **Hold and Wait:** A process holds one resource while requesting another.

3. **No Preemption:** Resources cannot be forcibly taken; they are released voluntarily.
4. **Circular Wait:** A circular chain exists where each process waits for a resource held by next process.

Example:

- P1 holds R1 and waits for R2.
- P2 holds R2 and waits for R1. Both wait forever, creating deadlock.

Handling strategies:

- Prevention (break one Coffman condition),
- Avoidance (safe-state algorithms like Banker's algorithm),
- Detection and recovery (detect cycles, then terminate/rollback processes).

Ans5. Virtual Memory and Paging

Virtual memory is a memory management technique that allows processes to use more memory than physically available RAM by using secondary storage (disk/SSD) as an extension. It provides each process with a large logical address space, improving multiprogramming and isolation.

In **paging**:

- Logical memory is divided into fixed-size **pages**.
- Physical memory is divided into fixed-size **frames**.
- OS maintains a **page table** mapping pages to frames.
- When required page is not in RAM, a **page fault** occurs and OS loads it from disk.

Benefits of paging:

- Eliminates need for contiguous memory allocation.
- Reduces external fragmentation.
- Supports demand paging (load pages only when needed).
- Improves memory utilization for large applications.

Costs:

- Address translation overhead,
- Page fault latency (disk access is slow),
- Possibility of thrashing if memory is overcommitted.

Overall, virtual memory + paging enables efficient and scalable execution of multiple programs.

Ans6. Synchronization, Critical Section, Mutex, Semaphore

In concurrent systems, multiple threads/processes may access shared data simultaneously. Without coordination, inconsistent data may result; this is known as a **race condition**.

A **critical section** is a code segment where shared resources are accessed and must be executed by only one thread/process at a time.

Synchronization ensures:

- mutual exclusion,
- correct ordering,
- data consistency.

Mechanisms:

Mutex:

- Binary lock with ownership.
- A thread locks before entering critical section and unlocks after exit.
- Only owner should unlock.
- Suitable for strict mutual exclusion in single-process multithreading.

Semaphore:

- Integer-based signaling primitive with wait(P) and signal(V) operations.
- **Binary semaphore** acts similar to mutex.
- **Counting semaphore** allows limited concurrent access to N instances of a resource.
- Common in producer-consumer and resource pool control.

Good synchronization avoids race conditions, deadlocks, and starvation while maintaining performance.

Ans7. Fragmentation: Internal vs External

Fragmentation is memory wastage caused by inefficient allocation patterns.

Internal fragmentation:

- Allocated memory block is larger than requested.
- Unused space remains inside allocated block.
- Common in fixed-size allocation systems.

External fragmentation:

- Free memory exists but is split into many small non-contiguous holes.
- A large contiguous request cannot be satisfied even if total free memory is enough.

Example:

- Internal: process needs 18 KB, allocator gives 20 KB; 2 KB wasted.
- External: free blocks of 5 KB, 7 KB, 8 KB cannot satisfy contiguous 15 KB request.

Reduction techniques:

- For internal fragmentation: use better-sized allocation units/slab classes.
- For external fragmentation: compaction, paging, buddy allocator strategies. Paging largely solves external fragmentation in main memory, though it may still have small internal waste in the last page.